

AppContextListener.java

```
package com.harisudhan.harisudhanmart.listener;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.Statement;
import java.util.Properties;
import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;
import javax.servlet.annotation.WebListener;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * Singleton pattern: owns the HikariCP connection pool lifecycle. No DriverManager.getConnection()
 * calls should exist anywhere outside this listener (Section 2, engineering rule 5).
 */
@WebListener
public class AppContextListener implements ServletContextListener {

    private static final Logger LOG = LoggerFactory.getLogger(AppContextListener.class);
    private static HikariDataSource dataSource;

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        Properties props = loadConfig();

        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(props.getProperty("db.url", "jdbc:h2:mem:harisudhanmart;DB_CLOSE_DELAY=-1"));
        config.setUsername(props.getProperty("db.user", "sa"));
        config.setPassword(props.getProperty("db.password", ""));
        config.setDriverClassName("org.h2.Driver");
        config.setMaximumPoolSize(Integer.parseInt(props.getProperty("db.pool.size", "10")));
        config.setPoolName("HarisudhanMartPool");

        dataSource = new HikariDataSource(config);
        LOG.info("HikariCP pool initialized against {}", config.getJdbcUrl());

        runSchema();
        sce.getServletContext().setAttribute("dataSource", dataSource);
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        if (dataSource != null) {
            dataSource.close();
            LOG.info("HikariCP pool closed");
        }
    }

    public static HikariDataSource getDataSource() {
        return dataSource;
    }

    private Properties loadConfig() {
        Properties props = new Properties();
        try (InputStream in = getClass().getClassLoader().getResourceAsStream("config.properties")) {
            if (in != null) {
                props.load(in);
            } else {
                LOG.warn("config.properties not found on classpath; using defaults (embedded H2)");
            }
        }
    }
}
```

```

    }
} catch (IOException e) {
    LOG.error("Failed to load config.properties", e);
}
return props;
}

/** Applies schema.sql on startup so a fresh embedded/server H2 instance is ready to use. */
private void runSchema() {
    try (Connection conn = dataSource.getConnection();
        InputStream in = getClass().getClassLoader().getResourceAsStream("schema.sql")) {
        if (in == null) {
            LOG.warn("schema.sql not found on classpath; skipping auto-init");
            return;
        }
        String sql = new String(in.readAllBytes());
        try (Statement stmt = conn.createStatement()) {
            for (String statement : sql.split(";")) {
                String trimmed = statement.trim();
                if (!trimmed.isEmpty()) {
                    stmt.execute(trimmed);
                }
            }
        }
        LOG.info("Schema applied successfully");
    } catch (Exception e) {
        LOG.error("Failed to apply schema.sql", e);
    }
}
}

```